Theory of Automata
CS-3005
Lecture 6

Mahzaib Younas

Lecturer Department of Computer Science

FAST NUCES CFD

Contents

- Unification
- Turning TGs into Regular Expressions
- Converting Regular Expressions into FAs
- Nondeterministic Finite Automata
- NFAs and Kleene’s Theorem

Unification

- We have learned three separate ways to define a language: (i) by **regular expression**, (ii) by **finite automaton**, and (iii) by **transition graph**.

- Now, we will present a theorem proved by Kleene in 1956, which says that **if a language can be defined by any one of these three ways, then it can also be defined by the other two**.

- In other words, Kleene proved that **all three of these methods of defining languages are *equivalent*.**

Theorem

- Any language that can be defined by **regular expression, or finite automaton, or transition graph** can be defined by all three methods.

Kleene Theorem

- This theorem is the most important and fundamental result in the theory of finite automata.
- We will take extreme care with its proofs. In particular, **we will introduce four algorithms that enable us to construct the corresponding machines and expressions.**
- Recall that
  - To prove A = B, we need to prove (i) $A \subset B$, and (ii) $B \subset A$.
  - To prove A = B = C, we need to prove (i) $A \subset B$, (ii) $B \subset C$, and (iii) $C \subset A$.

Kleene Theorem

- Thus, to prove Kleene's theorem, we need to prove 3 parts:

- **Part 1:** Every language that can be defined by a finite automaton can also be defined by a transition graph.

- **Part 2:** Every language that can be defined by a **transition graph can also be defined by a regular expression**.

- **Part 3:** Every language that can be defined by a regular expression can also be defined by a finite automaton.

Proof of Part 1: Every language that can be defined by a finite automaton can also be defined by a transition graph

- This is the easiest part.

- From previous lecture, we know that every finite automaton is itself already a transition graph. Therefore, any language that has been defined by a finite automaton has already been defined by a transition graph.

Proof of Part 2:

Every language that can be defined by a transition graph can also be defined by a regular expression.

Basic

- We prove this part by providing a **constructive algorithm**:

We present an algorithm that starts out with a transition graph and ends up with a regular expression that defines the same language.

To be acceptable as a method of proof, the algorithm we present will satisfy two criteria:

(i) It works for **every conceivable TG**, and

(ii) It guarantees to **finish its job in a finite number of steps.**

Creating a Unique Start State

Consider an abstract transition graph T that may have many start states.

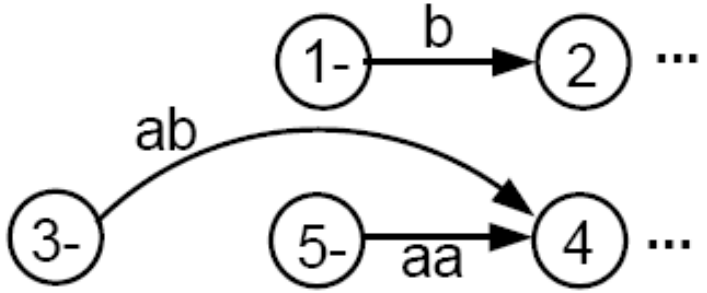
- We can simplify T so that it has **only one unique start state that has no incoming edges**.

- We do this by introducing a new start state that we label with the minus sign, and that we connect to all the previous start states by edges labeled with Λ. We then drop the minus signs from the previous start states.

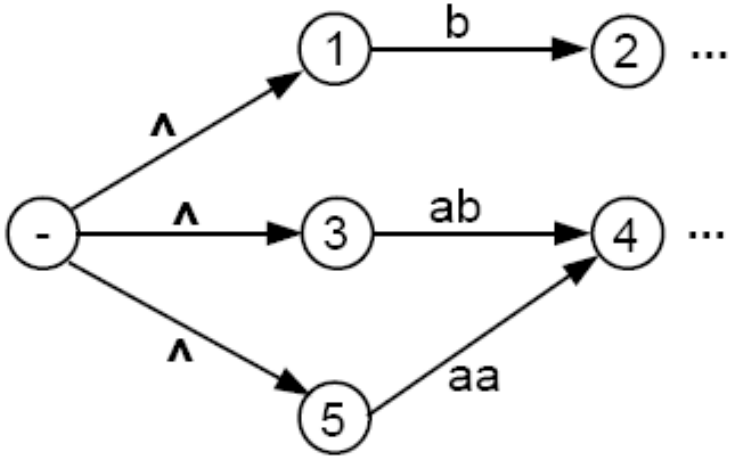
- If a word w used to be accepted by starting at one of the previous start states, then it can now be accepted by starting at the new unique start state.
- The following figure illustrates this idea.

Creating a Unique Start State

- Consider a fragment of T:



- The above fragment of T can be replaced by



Step 2: Creating a unique final State

- Let us make another simplification in T so that it has a **unique, un-exitable final state**, without changing the language it accepts.

- If T had no final state, then it accepts no strings at all and has no language. So, we need to produce no regular expression other than the null, or empty, expression Φ

- If T has several final states, we can introduce a new unique final state labeled with a plus sign. We then draw new edges from all the former final states to the new one, dropping the old plus signs, and labeling each new edge with the null string.
- This process is depicted in the next slide.

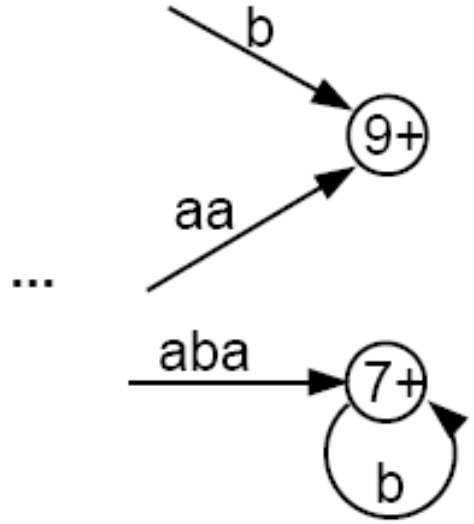
Step 2: Creating a unique final State

- Let us make another simplification in T so that it has a **unique, un-exitable final state**, without changing the language it accepts.

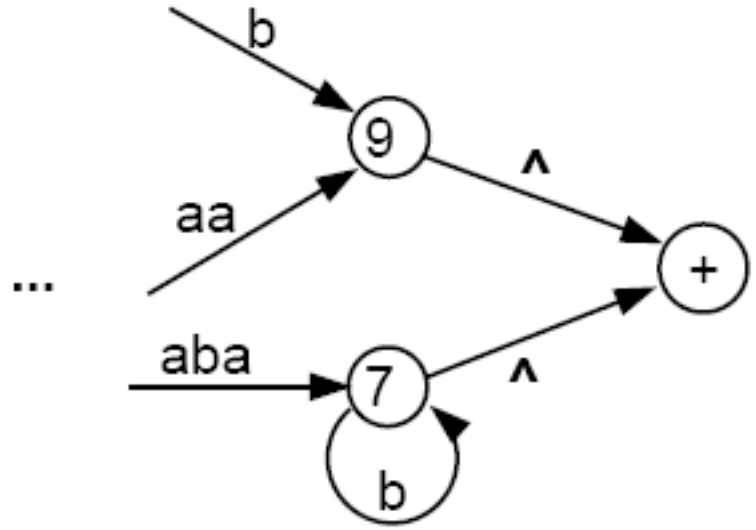
- If T had no final state, then it accepts no strings at all and has no language. So, we need to produce no regular expression other than the null, or empty, expression Φ

- If T has several final states, we can introduce a new unique final state labeled with a plus sign. We then draw new edges from all the former final states to the new one, dropping the old plus signs, and labeling each new edge with the null string.
- This process is depicted in the next slide.

Step 2: Creating a unique final State



Before



After

By applying step 1 and step 2

- We shall require that the unique final state be a different state from the unique start state. If an old state used to have ±, then both signs are removed from the old state to newly created states, using the processes described above.

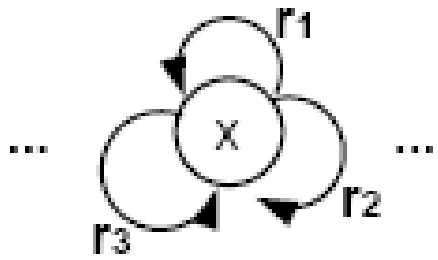
- It should be clear that the addition of these two new states does not affect the language that T accepts.

- The machine now has the following shape:

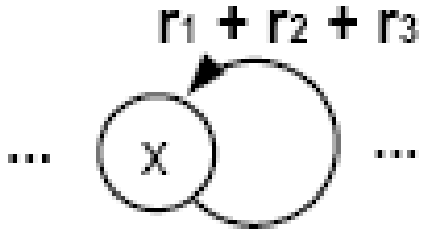
where there are no other - or + states.

Step 3 -- Combining Edges

- If T has some internal state x (not the - or the + state) that has more than one loop circling back to itself:

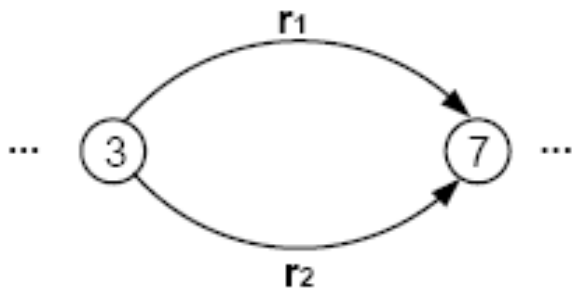


- where r1, r2, and r3 are all regular expressions or simple strings.
- We can replace the three loops by one loop labeled with a regular expression:

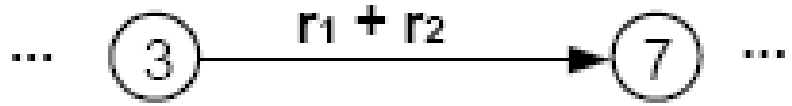


Step 3 -- Combining Edges

- Similarly, if two states are connected by more than one edge going in the same direction:

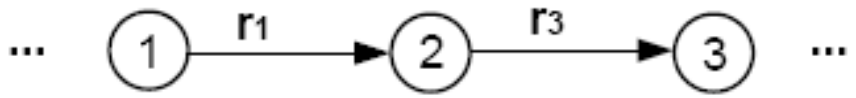


- We can replace this with a single edge labeled with a regular expression:

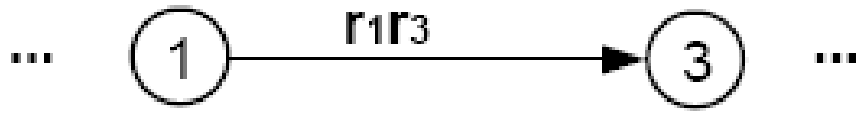


Step 4: Bypass and State Elimination

- If T has 3 states in a row connected by edges labeled with regular expressions or simple strings, we can eliminate the middle state, as in the following examples:

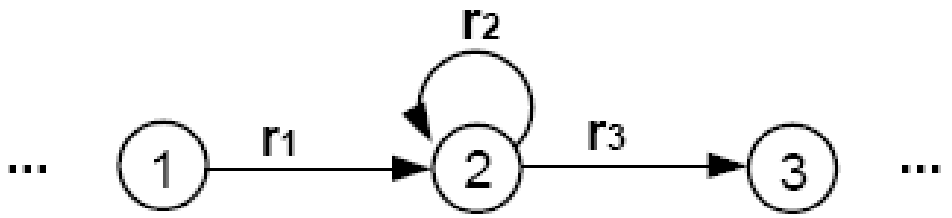


Can be replaced with

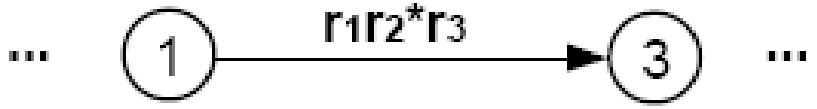


Step 4: Bypass and State Elimination

- If the middle state has a loop, we can proceed as follows:

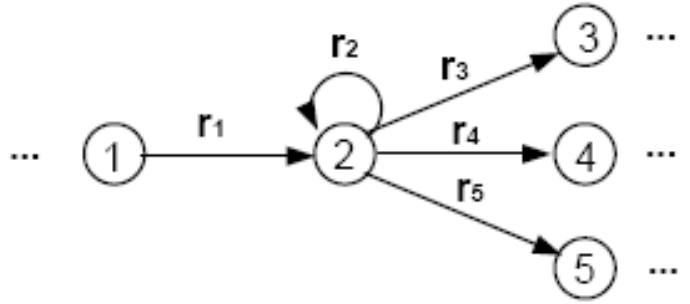


Can be replaced with

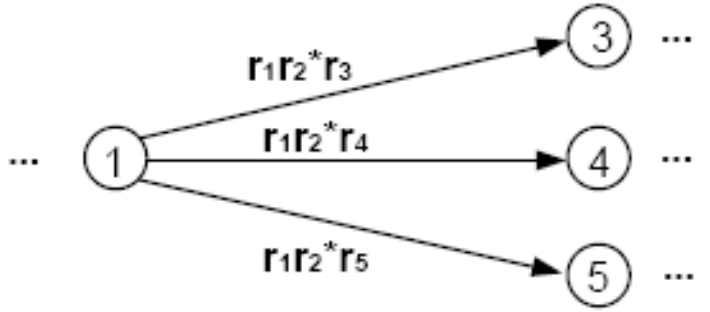


Step 4: Bypass and State Elimination

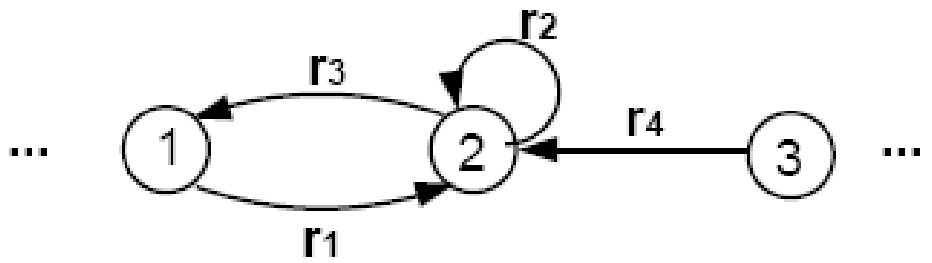
- If the middle state is connected to more than one state, then the bypass and elimination process can be done as follows:



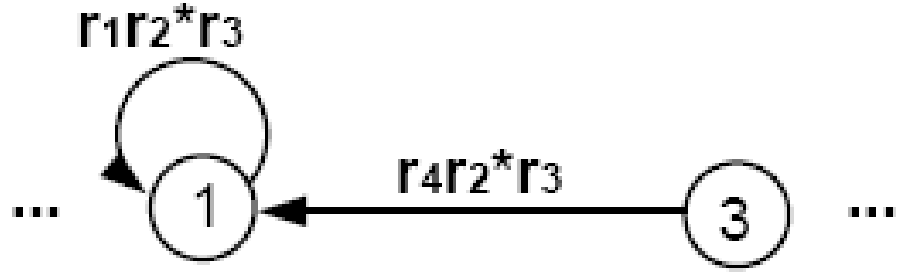
- Can be replaced with



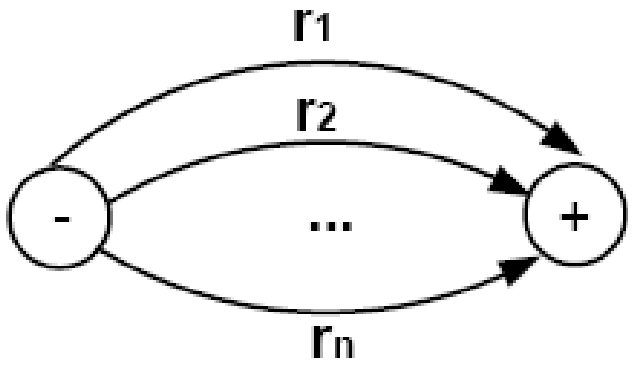
Special cases



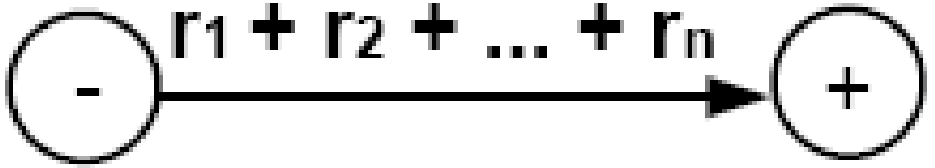
- Can be replaced with



- We can repeat this bypass and elimination process again and again until we have eliminated all the states from T, except for the unique start state and the unique final state. What we come down to is a picture that looks like this:

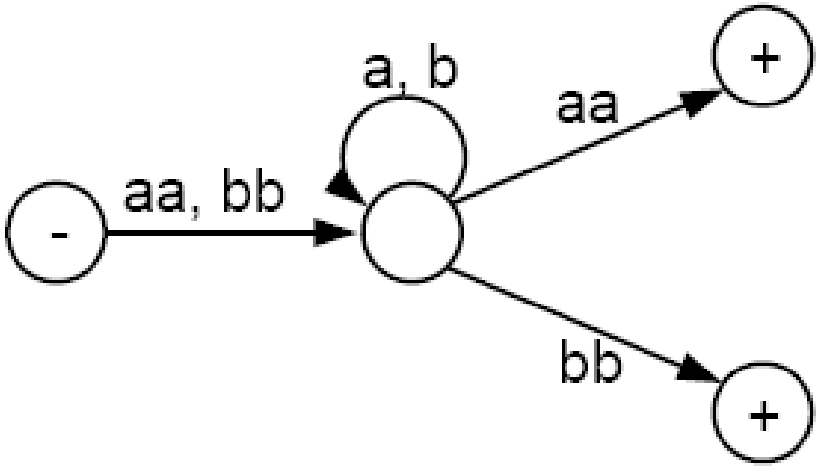


with each edge labeled by a regular expression.



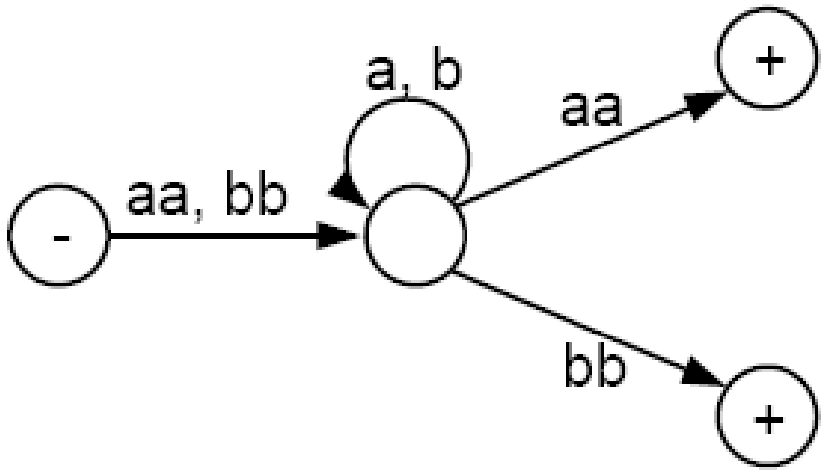
Example:

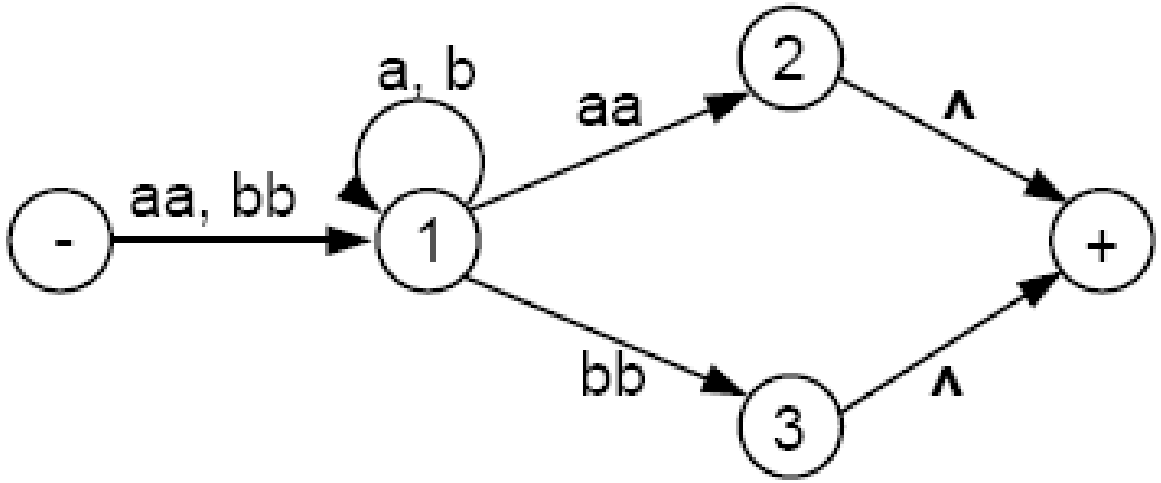
- Consider the following TG that accepts all words that begin and end with double letters (having at least length 4):



Step 1: Unique Final State

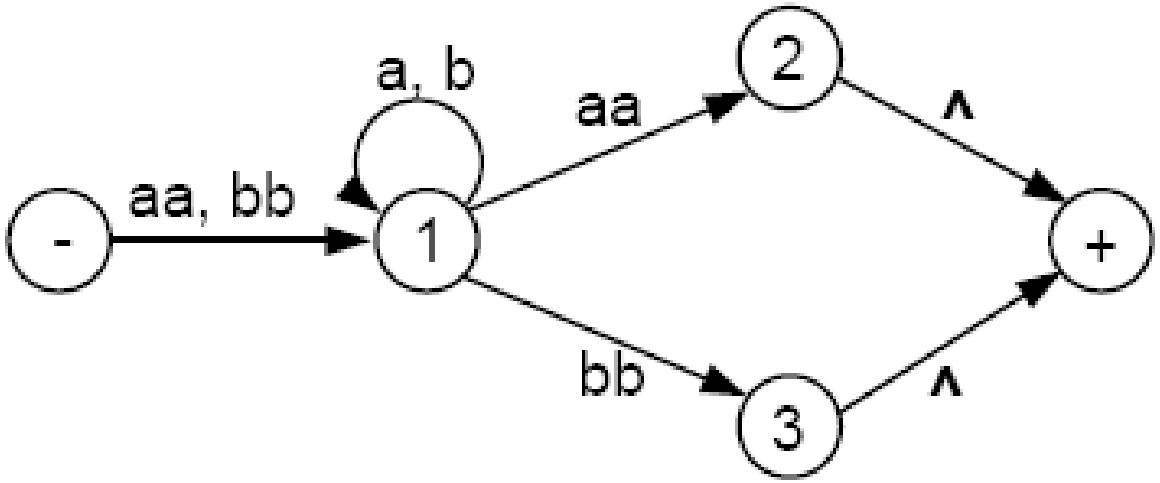
- This TG has only one start state with no incoming edges, but has two final states. So, we must introduce a new unique final state:

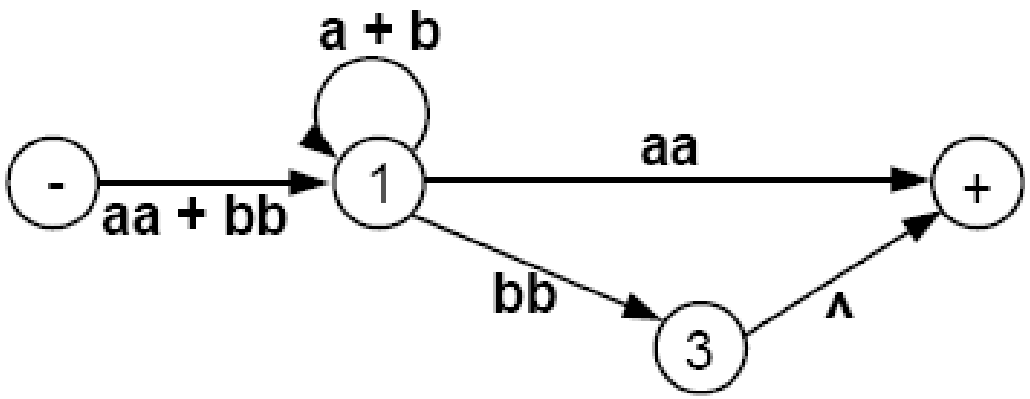




Step 2: Eliminate Second State

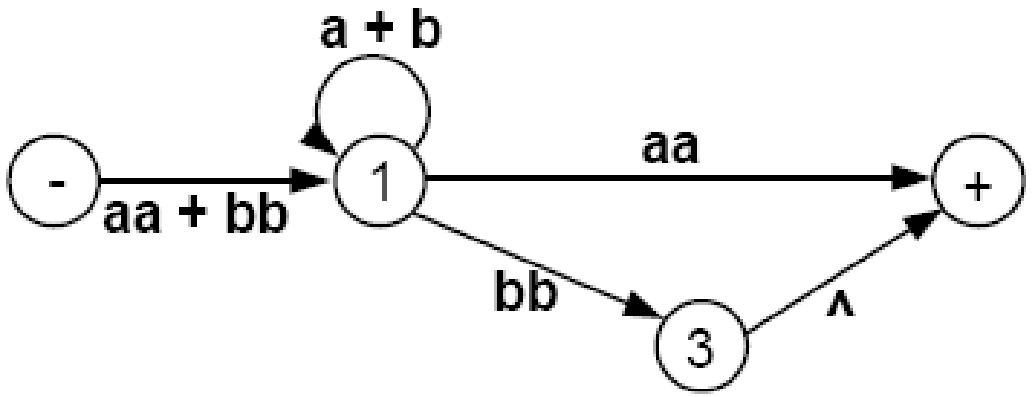
- Eliminate state 2:

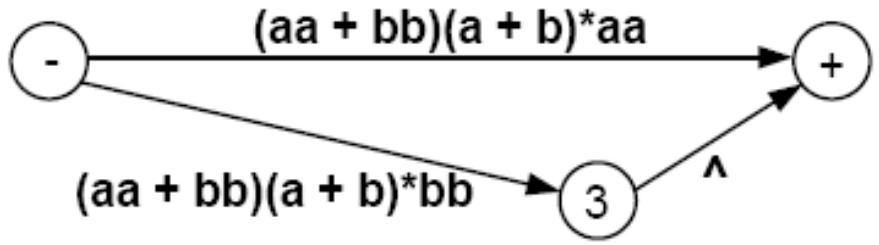




Step 3: Eliminate State 1

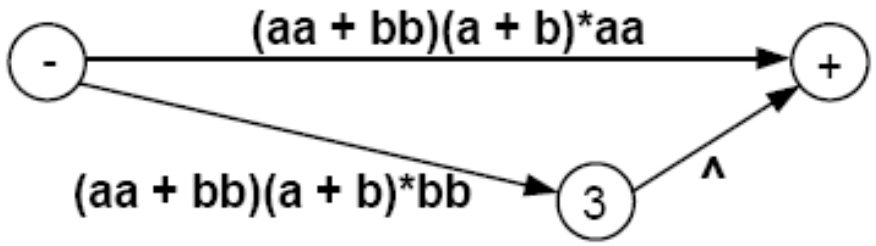
- Eliminate state 1

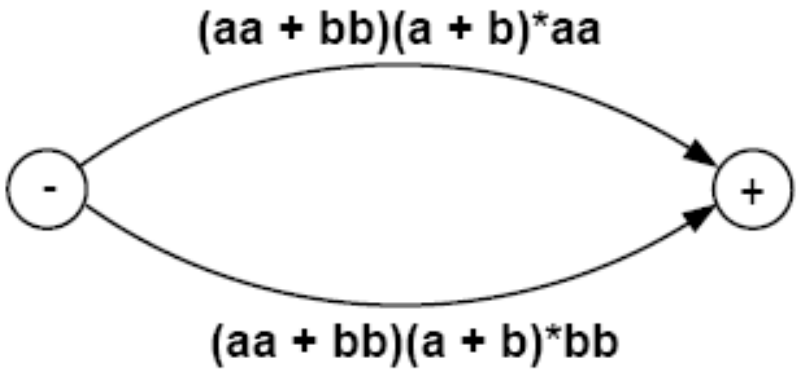




Step 3: Eliminate State 3

- Eliminate state 3





- Hence, this TG defines the same language as the regular expression

  (aa + bb)(a + b)*(aa) + (aa + bb)(a + b)*(bb)

- or equivalently

  (aa + bb)(a + b)*(aa + bb)

- If we eliminated the states in a different order, we could end up with a different-looking regular expression. But by the logic of the elimination process, **these expressions would all have to represent the same language**.

- We are now ready to present the constructive algorithm that proves that all **TGs can be turned into regular expressions** that define the exact same language.

Algorithm

- Step 1: Create a unique, unenterable minus state and a unique, unleaveable plus state.
- Step 2: One by one, in any order, bypass and eliminate all the non-minus or non-plus states in the TG. A state is bypassed by connecting each incoming edge with each outgoing edge. The label of each resultant edge is the concatenation of the label on the incoming edge with the label on the loop edge (if there is one) and the label on the outgoing edge.
- Step 3: When **two states are joined** by more than one edge going in the same direction, unify them by adding their labels.
- Step 4: Finally, when **all that is left is one edge from - to +, the label on that edge is a regular expression** that generates the same language as was recognized by the original TG.

Proof of Part 3:
Converting Regular Expressions into FAs

Basics

- Before we proceed, let's have a quick review of the **formal definition of regular expressions**.
- The set of **regular expressions** is defined by the following rules:
  - Rule 1: Every letter of the alphabet $\sum$ can be made into a regular expression by writing it in **boldface: Λ** itself is a regular expression.
  - Rule 2: If r_1 and r_2 are regular expressions, then so are:
    - (r_1)
    - r_1r_2
    - $r_1 + r_2$
    - r_1*
  - Rule 3: Nothing else is a regular expression.

We now present proof of part 3 recursively.

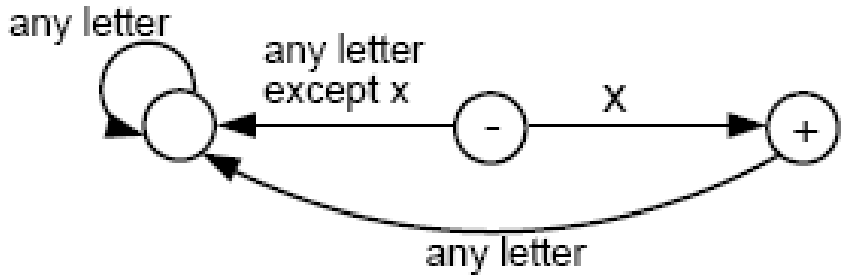
Rule 1:

- There is an FA that accepts any particular letter of the alphabet.

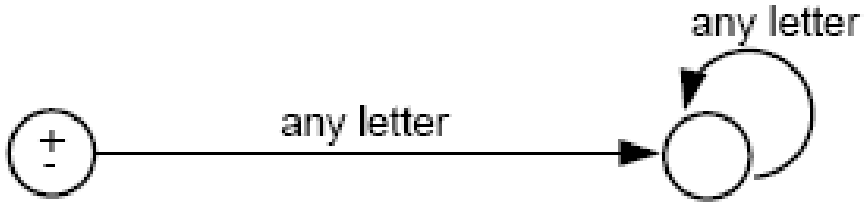
- There is an FA that accepts only the word Λ.

Proof of Rule 1:

- If letter x is in ∑, then the following FA accepts only the word **x**.



- The following FA accepts only **λ**:



Rule 2

- If there is an FA called FA_1 that accepts the language defined by the regular expression r_1, and there is an FA called FA_2 that accepts the language defined by the regular expression r_2, then there is an FA that we shall call FA_3 that accepts the language defined by the regular expression $(r_1 + r_2)$.

Algorithm for proof

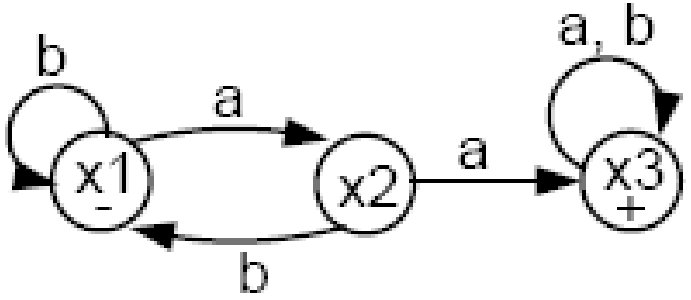
– To go from one state z to another by reading a letter from the input string, we observe what happens to the x part and what happens to the y part and go to the new state z accordingly. We could write this as a formula:

z_{new} after reading letter p = (x_{new} after reading letter p on FA_1) or (y_{new} after reading letter p on FA_2)

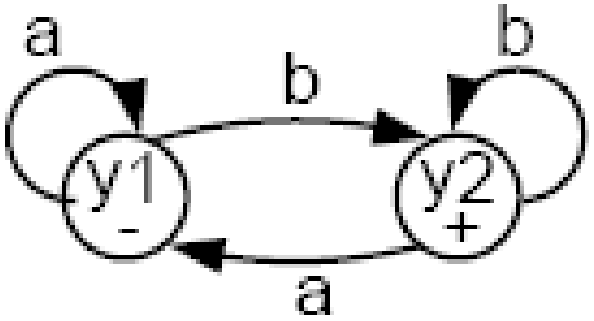
Proof

	a	**b**
z1=x1,y1	z2=x2,y1	z3=x1,y2
z2=x2,y1	z4=x3,y1	z3=x1,y2
z3=x1,y2	z2=x2,y1	z3=x1,y2
z4=x3,y1	z4=x3,y1	z5=x3,y2
z5=x3,y2	z4=x3,y1	z5=x3,y2

FA1

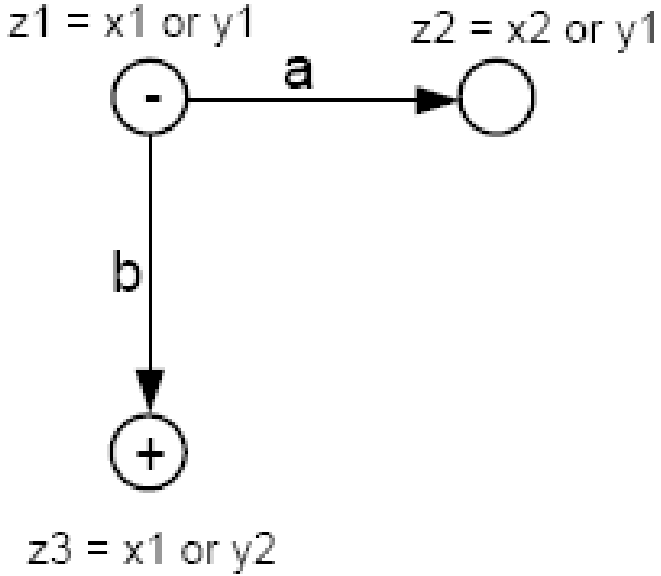


FA2



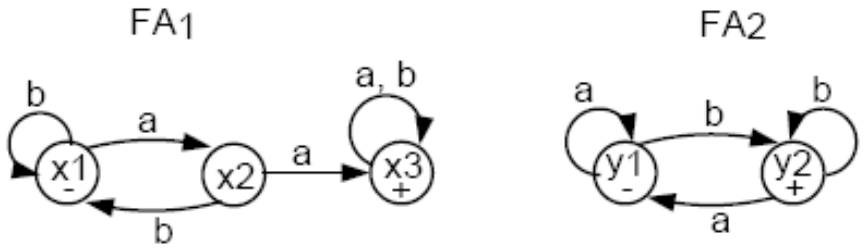
FA_1 accepts all words with a double a in them.
FA_2 accepts all words ending with b.
Let's follow the algorithm to build FA_3 that accepts the union of the two languages.

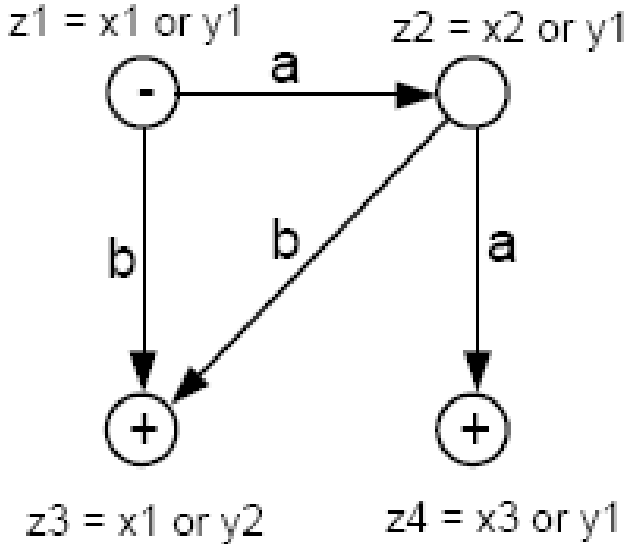
- The start (-) state of FA_3 is $z_1 = x_1$ or y_1.
- In z_1, if we read an ***a***, we go to x_2 (observing FA_1), or we go to y_1 (observing FA_2). Let $z_2 = x_2$ or y_1.
- In z_1, if we read a ***b***, we go to x_1 (observing FA_1), or to y_2 (observing FA_2). Let $z_3 = x_1$ or y_2. Note that z_3 must be a final state since y_2 is a final state.



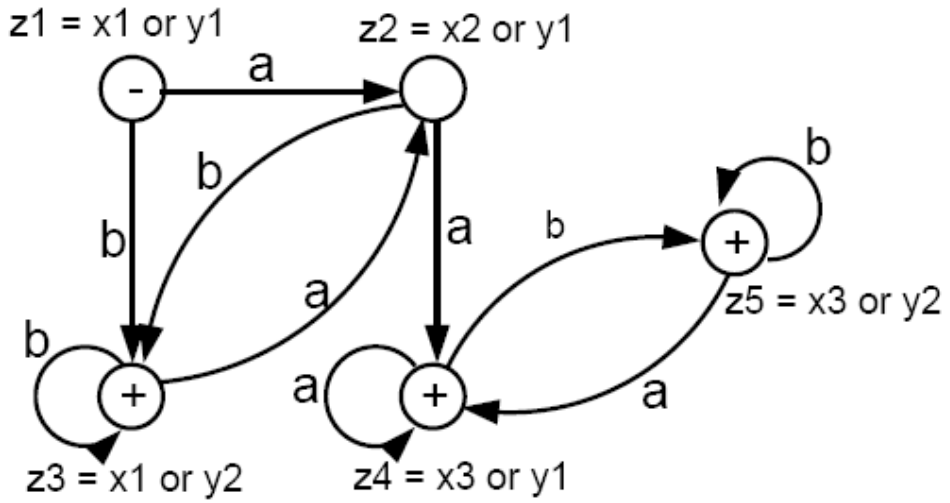
By combining FA1 and FA2

- In z_2, if we read an a, we go to x_3 or y_1. Let $z_4 = x_3$ or y_1. z_4 is a final state because x_3 is.
- In z_2, if we read a b, we go to x_1 or y_2, which is z_3.





Combining two Machines



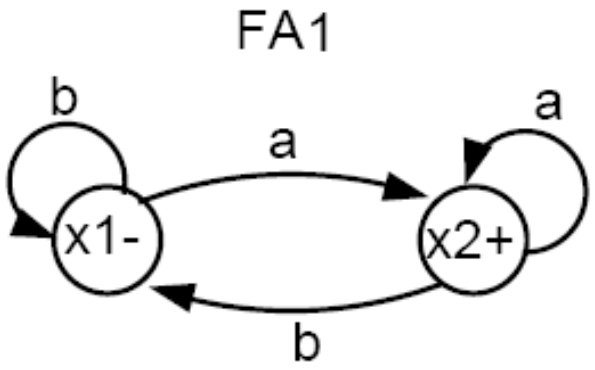
This machine accepts all words that have a double *a* or that end with *b*.
The labels $z_1 = x_1$ **or** y_1, $z_2 = x_2$ **or** y_1, etc. can be removed if you want.

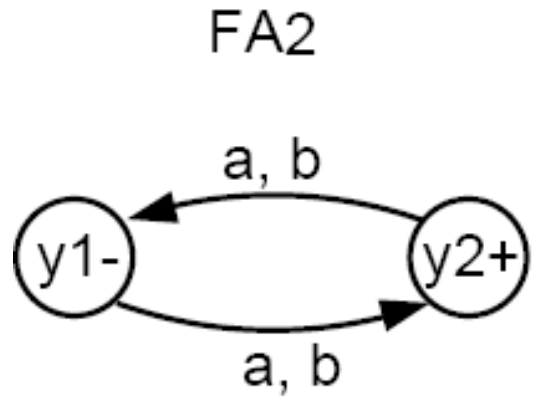
Details:

- In z_3, if we read an a, we go to x_2 or y_1, which is z_2.
- In z_3, if we read a b, we go to x_1 or y_2, which is z_3.
- In z_4, if we read an a, we go to x_3 or y_1, which is z_4. Hence, we have an a-loop at z_4.
- In z_4, if we read a b, we go to x_3 or y_2. Let $z_5 = x_3$ or y_2. Note that z_5 is a final state because x_3 (and y_2) are.
- In z_5, if we read an a, we go to x_3 or y_1, which is z_4.
- In z_5, if we read a b, we go to x_3 or y_2, which is z_5. Hence, there is a b-loop at z_5.
- The whole machine looks like the following:

Example:

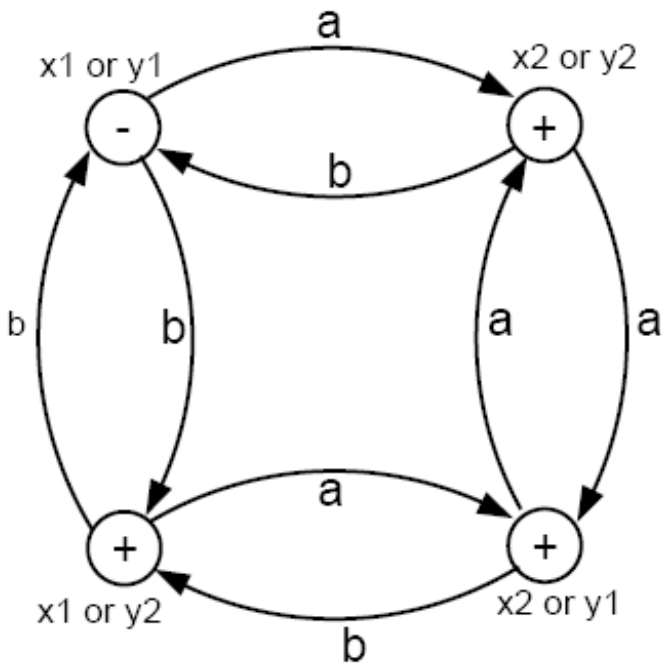
- Consider the following two FAs:





- FA_1 accepts all words that end in *a*.
- FA_2 accepts all words with an odd number of letters (odd length).
- Can you use the algorithm to build a machine FA3 that accepts all words that either have an odd number of letters or end in *a*?

- Using the algorithm, we can produce FA_3 that accepts all words that either have an odd number of letters or end in a, as follows:



- The only state that is not a + state is the - state. To get back to that start state, a word must have an even number of letters **and** end in *b*.

Rule 3

- If there is an FA_1 that accepts the language defined by the regular expression r_1, and there is an FA_2 that accepts the language defined by the regular expression r_2, then there is an FA_3 that accepts the language defined by the (concatenation) regular expression $(r_1 r_2)$, i.e. the product language.

Algorithm

- First, create a state *z* for every state of FA_1 that we may go through before arriving at a final state.

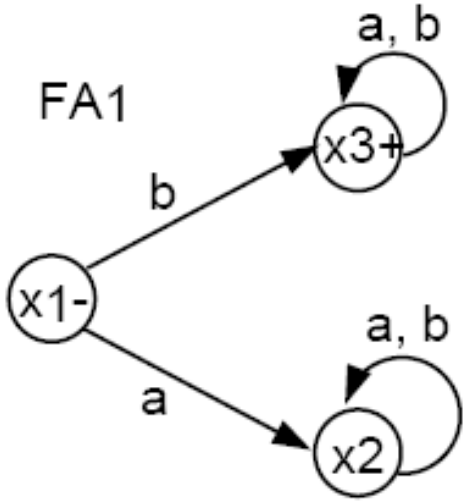
2. For each final state x_{final} of FA_1, add a state $z = x_{final}$ or y_1, where y_1 is the start state of FA_2.

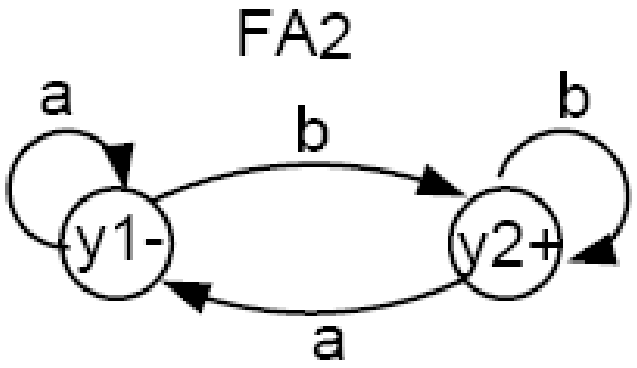
3. From the states added in step 2, add states

$$z = \begin{cases} x_{something} \text{ indicating that we are still continuing on } FA_1 \\ \text{OR} \\ \text{a set of } y_{something} \text{ indicating that we are on } FA_2 \end{cases}$$

4. Label every state *z* that contains a final state from FA_2 as a final state.

Example:

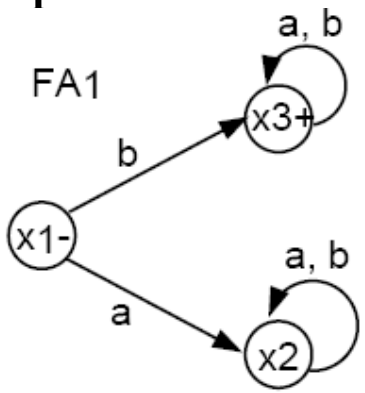


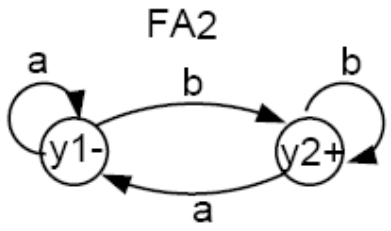


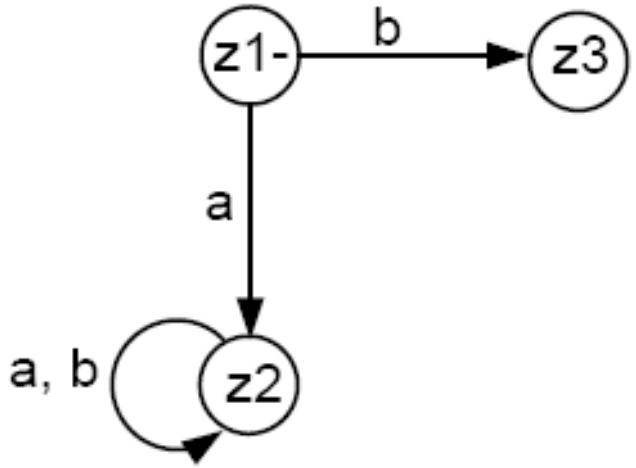
- FA1 accepts all words that start with a b.
- FA2 accepts all words that end with a b.
- We will use the above algorithm to construct FA3 that accepts the product of the languages of FA1 and FA2, respectively. That is, FA3 will accept all words that both start and end with the letter b.

Example:

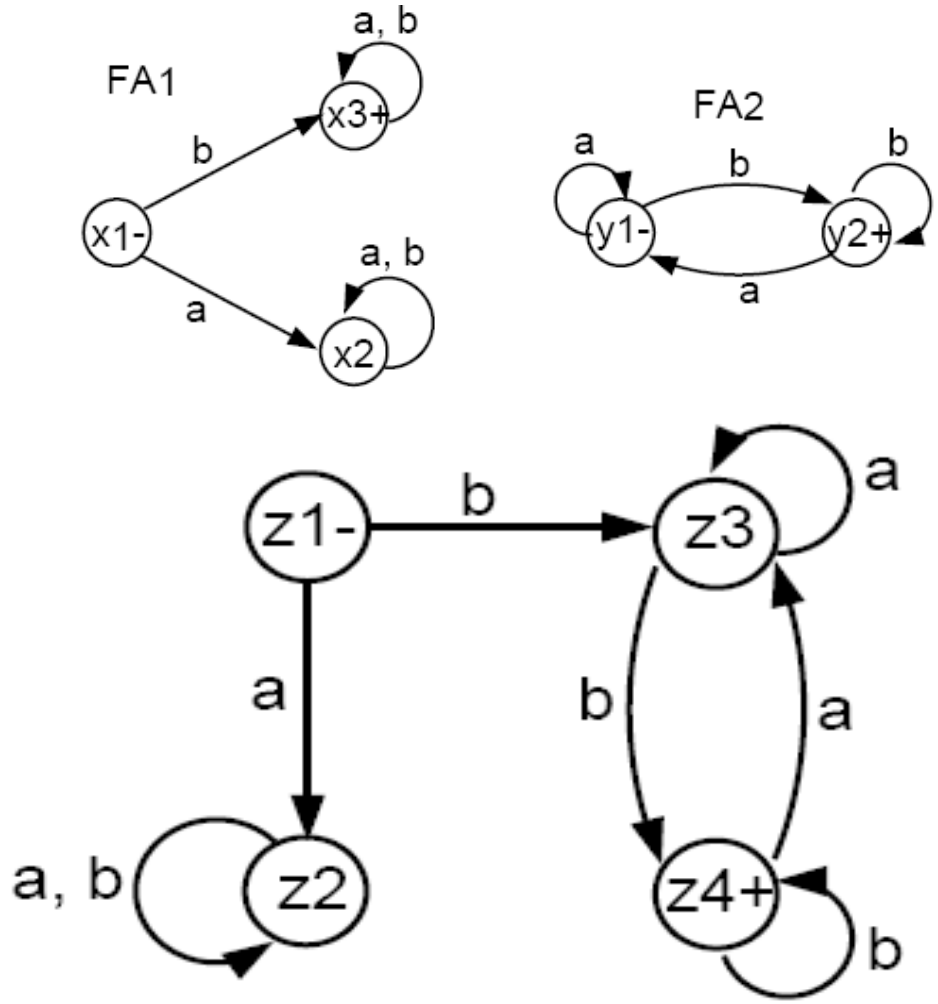
- Initially, we must begin with $x_1 = z_1$.
- In z_1, if we read an a, we go to $x_2 = z_2$.
- In z_1, if we read a b, we go to x_3, a final state, which gives us the option to jump to y_1. Hence, we label $z_3 = x_3$ or y_1.
- From z_2 just like x_2, both an a or a b take us back to z_2, i.e., we have a loop here.







	a	b
z3=x3,y1	z3=x3,y1	z4=x3,y2
z4=x3,y2	z3=x3,y1	z4=x3,y2



- This machine accepts all words that both begin and end with the letter b, which is what the product of the two languages (defined by FA_1 and FA_2 respectively) would be.

- If you multiply the two languages in opposite order (i.e. first FA_2 then FA_1), then the product language will be different. What is that language? Can you build a machine for that product language